Retail Microservices System

The Complete Beginner's Guide

Analysis · Architecture · Design · Implementation · Security · Deployment · Testing

Every concept used to build the project, explained in plain language, with the reason it was chosen.

Stack: Java 17 · Spring Boot 3.5 · Spring Cloud 2025 · MySQL 8 · Apache Kafka · Docker · Elasticsearch/Logstash/Kibana · HashiCorp Vault · GitHub Actions

Contents

1. How to read this guide **5**

1.1 What you will learn . . . 5

1.2 How the guide is organised . . . 5

2. Analysis: what are we building? **6**

2.1 The business problem . . . 6

2.2 Actors (who uses the system) . . . 6

2.3 Functional requirements (what it must do) . . . 6

2.4 Non-functional requirements (how well it must do it) . . . 6

2.5 Constraints and assumptions . . . 7

2.6 Core business rules we had to get right . . . 7

3. Foundations: the vocabulary of the project **8**

4. Architecture **11**

4.1 The big picture . . . 11

4.2 The services at a glance . . . 11

4.3 How a request travels . . . 12

4.4 Communication styles: synchronous and asynchronous . . . 12

4.5 Folder layout of the repository . . . 12

5. Design decisions and patterns (and why) **14**

5.1 Centralised configuration (Spring Cloud Config) . . . 14

5.2 Service discovery (Eureka) . . . 14

5.3 API Gateway . . . 14

5.4 Stateless authentication with JWT (RS256) . . . 14

5.5 Role-based access control (RBAC) . . . 15

5.6 Database per service . . . 15

5.7 Idempotency . . . 16

5.8 Concurrency control: locks and conditional updates . . . 16

5.9 The Saga pattern (distributed transaction) . . . 16

5.10 Transactional outbox . . . 16

5.11 Resilient synchronous calls (Feign + Resilience4j) . . . 17

5.12 Soft delete and audit . . . 17

5.13 Field-level encryption (AES converter) . . . 17

5.14 Shared library (common-lib) and auto-configuration . . . 18

6. Implementation: service by service **19**

6.1 The anatomy of a typical service . . . 19
6.2 auth-service (port 8081) . . . 19
6.3 customer-service (8082) . . . 20
6.4 product-service (8083) . . . 20
6.5 inventory-service (8084) . . . 21
6.6 cart-service (8085) . . . 21
6.7 campaign-service (8089) . . . 21
6.8 order-service (8086): see chapter 7 . . . 21
6.9 payment-service (8087) . . . 21
6.10 review-service (8088) . . . 22
6.11 wishlist-service (8091) . . . 22
6.12 support-service (8090) . . . 22
6.13 notification-service (8092) . . . 22

7. Deep dive: the order saga 23

7.1 The happy path . . . 23
7.2 Why the database work is split into small transactions . . . 23
7.3 Reconcilers: the safety net . . . 24
7.4 Order status lifecycle . . . 24
7.5 Worked numbers . . . 24

8. Cross-cutting concerns in common-lib 25

8.1 Uniform error responses . . . 25
8.2 Input validation . . . 25
8.3 Structured logging (ECS JSON) and correlation . . . 25
8.4 Audit logging . . . 25
8.5 Actuator health and metrics . . . 25

9. Production hardening: TLS, secrets, encryption 26

9.1 TLS and mutual TLS between services . . . 26
9.2 Secrets and encrypted configuration . . . 26
9.3 Protecting internal endpoints . . . 26
9.4 Other security details . . . 27

10. Running everything: Docker, Compose and observability 28

10.1 The Dockerfile . . . 28
10.2 docker-compose.yml . . . 28
10.3 Observability: logs, traces, metrics . . . 28

11. CI/CD: automatic build, test, scan and deploy 30

12. Testing **31**

12.1 The testing pyramid . . . 31

12.2 Unit tests we wrote and what they prove . . . 31

12.3 Testing with Postman (manual and automated) . . . 31

12.4 Testing failure paths on purpose . . . 32

12.5 What to check in the data . . . 32

13. Real problems we hit (and what they teach) **33**

14. Step-by-step: run it yourself **34**

14.1 Prerequisites . . . 34

14.2 Option A: run from IntelliJ (development) . . . 34

14.3 Option B: Docker Compose (whole stack) . . . 34

14.4 Handy URLs . . . 34

15. Honest limitations and next steps **35**

16. Glossary **36**

1. How to read this guide

This document explains a complete online-shop back end, built as many small cooperating programs (called **microservices**). It is written for someone who has seen a little Java but has never built a distributed system. Every new word is explained the first time it appears, and each important design choice has a **Why we did this** box so you learn the reasoning and not just the recipe.

1.1 What you will learn

- How to turn a business requirements document (BRD) into a list of services and data.
- How the pieces talk to each other, and what happens when one of them breaks.
- How login, passwords and tokens work, and how the system is locked down (TLS, secrets, encryption).
- How a customer order moves safely through stock, payment and delivery (the *saga*).
- How the whole stack is started with one command, monitored, tested and shipped automatically.
- How to test it all, including a ready-made Postman collection.

1.2 How the guide is organised

Part	Chapters	What it answers
Analysis	2	What are we building and for whom?
Foundations	3	The vocabulary and tools (REST, Spring, JPA, Docker...)
Architecture & design	4, 5	The big picture and every pattern we used and why
Implementation	6, 7, 8	Each service, the order saga in depth, shared code
Security & operations	9, 10, 11	TLS, Vault, Docker Compose, logging, CI/CD
Testing & troubleshooting	12, 13, 14	How to test, real bugs we hit, how to run it
Reference	15, 16	Limitations and glossary

Beginner tip
Read chapters 2 to 5 first even if you only care about code. Once you understand the picture, the code in chapters 6 to 8 reads like a story instead of a pile of files.

Note / limitation
Code snippets in this guide are **simplified** so they fit on a page and teach one idea. The real files in the repository are longer (validation, logging, edge cases). Field names in the sample requests follow the project's DTOs as designed; if a request returns HTTP 400, compare it with the DTO class.

2. Analysis: what are we building?

2.1 The business problem

A retail company wants to sell products online. Customers must be able to browse a catalogue, keep items in a basket, pay, and track delivery. Staff need to manage products, stock, discount campaigns, orders and customer support tickets. The Business Requirements Document (BRD) lists these needs. Software analysis means reading the BRD and turning every sentence into something concrete: who uses the system, what they can do, what data must be stored, and what rules must hold.

2.2 Actors (who uses the system)

Actor	Description	Typical actions
Guest	Visitor who has not logged in	Browse products, search, keep a basket (cart token), register, log in
Customer	Registered, logged-in shopper	Manage profile and addresses, checkout, track and cancel orders, write reviews, keep a wishlist, open support tickets
Admin / staff	Back-office user (role ADMIN)	Manage categories, products, stock, campaigns; change order status; moderate reviews; answer support tickets; view notifications
System	Background jobs and other services	Send emails, release stuck stock, retry failed messages, sync ratings

2.3 Functional requirements (what it must do)

Module	Requirement summary
Authentication	Register/login, short-lived access tokens, refresh tokens, logout, lock account after repeated failures, roles
Customer	Profile, multiple addresses, one default address
Product catalogue	Categories, products, search and filters (text, category, brand, price, rating, sort)
Inventory	Stock per product, reserve stock during checkout, history of every change
Cart	Guest and customer carts, add/update/remove, merge guest cart on login
Order	Place order exactly once, price calculation (tax, shipping, discounts), status lifecycle, tracking, cancellation
Payment	Charge a card token; handle declines, insufficient funds, timeouts
Review & rating	Only buyers of a delivered product can review; average rating shown on the product
Campaign	Time-boxed percentage/fixed discounts that automatically start and end
Wishlist	Save products, move them to the cart
Support	Customers open tickets, agents reply, assign, resolve
Notification	Emails for registration, order confirmation, status change, support replies, low stock

2.4 Non-functional requirements (how well it must do it)

These are qualities rather than features. They drive most architectural decisions, which is why they matter so much.

Quality	Meaning	How we met it
Scalability	Handle more users by adding copies of the busy parts	Independent services, stateless JWT login, Kafka
Reliability	A failing part must not corrupt data or take everything down	Circuit breakers, retries, saga, outbox, reconcilers
Consistency	No double charge, no overselling	Idempotency keys, row locks, conditional updates
Security	Only the right people see the right data; secrets are protected	JWT RS256, role checks, TLS, Vault, AES-encrypted fields
Observability	We can see what happened and why	Structured logs, ELK, tracing (Zipkin), audit log
Maintainability	Teams can change one part safely	Config server, shared library, database migrations, CI/CD
Portability	Runs the same everywhere	Docker and Docker Compose

2.5 Constraints and assumptions

- Java 17 language level (the developer machine uses a newer JDK, which still runs Java 17 code).
- MySQL 8 database, one separate database per service.
- Windows + IntelliJ + PowerShell for development; Docker Desktop for the full stack.
- Payments are simulated with special tokens (tok_ok_4242, tok_declined, tok_insufficient, tok_timeout); no real card network is called.
- Email goes to MailHog (a fake mail server) in development.

2.6 Core business rules we had to get right

1. **Never oversell.** Two customers cannot buy the last item.
2. **Never double charge.** If the customer's browser retries the checkout, the order is created once.
3. **Money maths is exact.** We use decimal numbers (BigDecimal), never floating point. Example: 19.99 x 2 + 24.50 = 64.48 subtotal; 10% tax = 6.45; shipping 5.00 (free from 100); total 75.93.
4. **One review per purchase**, and only after delivery.
5. **Nothing is truly deleted.** Records are marked deleted (soft delete) so history and audits survive.

3. Foundations: the vocabulary of the project

Before the architecture, here are the building blocks. Each entry says what it is, with an everyday analogy, and why the project needs it.

3.1 Monolith vs microservices

What: A monolith is one big program containing every feature. Microservices split the system into many small programs, each owning one business area and talking over the network.

Analogy: A monolith is one giant department store with a single manager. Microservices are a shopping street of small specialist shops, each with its own owner.

Why here: The BRD has twelve clearly separate areas (orders, payments, reviews...). Splitting them lets teams build, deploy and scale each independently, and a bug in reviews cannot crash checkout. The price is complexity: networks fail, data is spread out. Much of this guide is about managing that price.

3.2 HTTP, REST and JSON

What: HTTP is the language of the web (GET to read, POST to create, PUT to update, DELETE to remove). REST is a style of designing URLs around nouns, like /api/v1/products/42. JSON is the text format of the data, like {"name":"Headphones"}.

Analogy: REST is like a library catalogue: every book has an address; verbs say what you do with it.

Why here: Every service exposes a REST API so web, mobile and other services can use it in a standard way. Status codes carry meaning: 200 OK, 201 Created, 400 bad input, 401 not logged in, 403 not allowed, 404 not found, 409 conflict, 5xx server problem.

3.3 Java, Maven and the multi-module project

What: Java is the language. Maven builds the project and downloads libraries listed in pom.xml files. A multi-module project has a parent pom plus one module per service.

Analogy: The parent pom is the master shopping list; each module's pom is a recipe that says 'use items from the master list'.

Why here: The parent pins library versions once (Spring Boot 3.5.8, Spring Cloud 2025.0.x) so every service uses compatible versions. One command (mvn clean install) builds everything.

3.4 Spring Boot

What: A framework that starts a web server and wires your classes together with almost no configuration. Key ideas: **annotations** (@RestController, @Service, @Entity), **dependency injection** (Spring creates objects and hands them to whoever needs them), **auto-configuration** (add a library, it configures itself), and **starters** (one dependency that pulls a whole feature).

Analogy: Like a restaurant kitchen where the head chef (Spring) assigns ingredients to the right cooks, so you only write recipes.

Why here: It removes boilerplate and is the industry standard for Java services. Each microservice is a small Spring Boot application with its own main class.

3.5 Layered code: Controller, Service, Repository

What: Controller receives HTTP and returns JSON. **Service** holds business rules. **Repository** talks to the database. DTOs (data transfer objects) are the shapes sent over HTTP; entities are the shapes stored in the database.

Analogy: Waiter (controller) takes the order, chef (service) cooks, pantry clerk (repository) fetches ingredients.

Why here: Separation keeps each class small and testable; business rules live in one place regardless of whether the call came from HTTP, Kafka or a scheduled job.

3.6 JPA, Hibernate and Spring Data

What: JPA lets you treat database rows as Java objects. Hibernate is the engine. Spring Data JPA generates repository methods such as findByEmail from their names.

Analogy: A translator between the language of objects and the language of tables.

Why here: Less hand-written SQL for ordinary reads and writes. Where exact control matters (atomic stock updates) we write explicit @Query SQL instead.

3.7 Flyway database migrations

What: SQL files named V1__init.sql, V2__... applied once, in order, and recorded in a flyway_schema_history table.

Analogy: Version control for your database structure, like git for tables.

Why here: Everyone, including Docker and CI, gets exactly the same schema. Hibernate is told not to create tables itself (it only validates) so the schema is never changed by accident.

3.8 Docker and Docker Compose

What: Docker packages a program with everything it needs into an **image**; a running copy is a **container**. Docker Compose starts many containers from one docker-compose.yml.

Analogy: A shipping container: whatever is inside, every ship and crane handles it the same way.

Why here: The system has about 18 containers (12 services plus MySQL, Kafka, Eureka, config server, gateway, MailHog, Vault, ELK...). One command starts all of them identically on any laptop.

3.9 Kafka and events

What: Apache Kafka is a durable message log. A service **publishes** an event (ORDER_CREATED) to a **topic**; other services **consume** it whenever they are ready.

Analogy: A noticeboard with a permanent record: the poster does not wait for readers, and late readers can still catch up.

Why here: Notification, review and support services react to what happened in other services without those services knowing about them. That keeps services loosely coupled.

3.10 JWT (JSON Web Token)

What: A signed text token proving who you are and what roles you have. It has three parts: header, payload (claims such as roles and customerId) and a signature.

Analogy: A festival wristband: staff glance at it instead of phoning the ticket office each time.

Why here: Services stay **stateless**: they verify the signature locally, with no shared session store.

3.11 Resilience: timeouts, retries and circuit breakers

What: Networks fail. A timeout stops waiting; a retry tries again; a circuit breaker stops calling a service that keeps failing and fails fast until it recovers.

Analogy: An electrical fuse: it trips to protect the house instead of letting wires burn.

Why here: If payment is slow, checkout should fail cleanly after 4 seconds, not hang and pile up threads until everything stalls.

4. Architecture

4.1 The big picture

The diagram below is the whole system on one page. Read it top to bottom: a client calls the **API Gateway**; the gateway checks the login token and forwards the request to the right **service**; services keep their data in their own **MySQL** databases and announce important facts through **Kafka**. Around the edge sit helper systems: a **Config Server** that hands out settings, **Eureka** that works as a phone book of running services, and the observability tools that let us see inside.

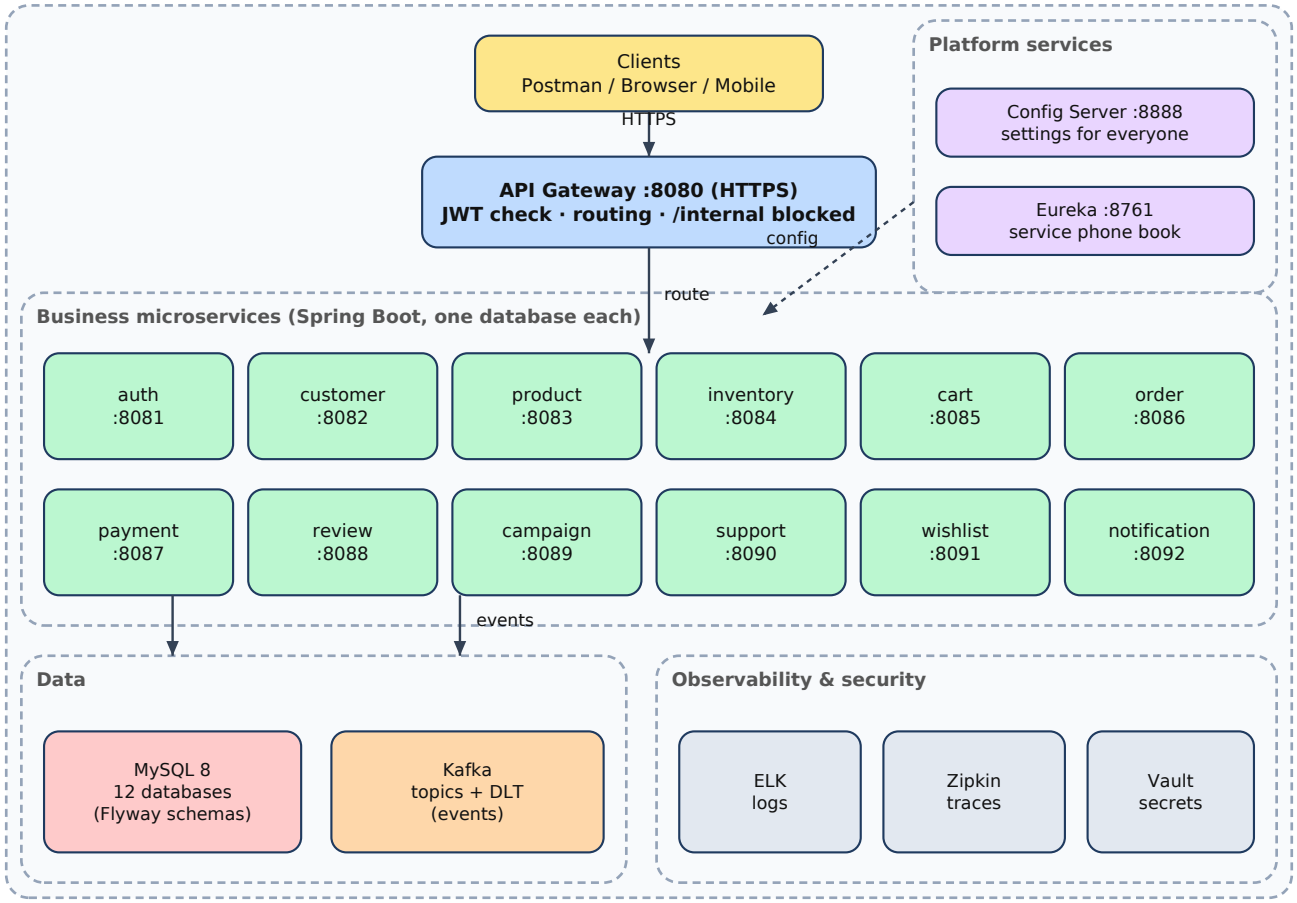


Figure 1 - System architecture

4.2 The services at a glance

Service	Port	Owns (database)	Responsibility in one line
api-gateway	8080	-	Single entry point; validates JWT; routes; hides /internal/**
config-server	8888	git repo	Central configuration for all services
eureka-server	8761	-	Registry: which service instances are alive and where
auth-service	8081	auth_db	Credentials, login, refresh tokens, JWT issuing, JWKS
customer-service	8082	customer_db	Profiles and addresses (sensitive fields encrypted)
product-service	8083	product_db	Categories, products, search, rating copy
inventory-service	8084	inventory_db	Stock levels, reservations, stock history
cart-service	8085	cart_db	Guest and customer carts, merge

Service	Port	Owns (database)	Responsibility in one line
order-service	8086	order_db	Order saga, pricing, status lifecycle, tracking
payment-service	8087	payment_db	Charges (simulated), idempotent per order
review-service	8088	review_db	Reviews and per-product rating summary
campaign-service	8089	campaign_db	Discount campaigns and the price engine
support-service	8090	support_db	Tickets, messages, agent workflow
wishlist-service	8091	wishlist_db	Saved products, move to cart
notification-service	8092	notification_db	Templates, email delivery, retries

4.3 How a request travels

1 The client sends `GET /api/v1/orders` to the gateway with `Authorization: Bearer <JWT>`.

2 The gateway verifies the token signature and expiry using the public key. If invalid: HTTP 401, the request goes no further.

3 The gateway matches the path to a route (order-service) and asks Eureka for a healthy instance (load balancing).

4 The order-service verifies the token again (defence in depth), reads claims such as customerId and roles, and applies its own rules.

5 The service reads or writes *its own* database. If it needs data owned by another service, it calls that service's API using Feign.

6 The response goes back through the gateway to the client.

> **Why we did this**
> Verifying the token in both gateway and service means that even if someone bypasses the gateway on the internal network, the service still refuses unauthenticated calls.

4.4 Communication styles: synchronous and asynchronous

Style	Technology	Use when	Example in this project
Synchronous (ask and wait)	REST via Feign	You need the answer now to continue	Order asks inventory to reserve stock; order asks payment to charge
Asynchronous (announce and move on)	Kafka events	Others merely need to know it happened	ORDER_CREATED to notification; rating changes to product

4.5 Folder layout of the repository

Repository structure

```
retail-microservices/
  pom.xml                  <- parent: versions, modules
  common-lib/              <- shared code (errors, security, outbox, audit, AES)
  config-server/ eureka-server/ api-gateway/
  auth-service/ customer-service/ product-service/ inventory-service/
  cart-service/ order-service/ payment-service/ review-service/
  campaign-service/ support-service/ wishlist-service/ notification-service/
  docker/                  <- mysql init, elk, tls, vault
  docker-compose.yml  docker-compose.tls.yml  docker-compose.vault.yml
  ci/                      <- build, scan, smoke-test, deploy scripts
  .github/workflows/ci-cd.yml
retail-config-repo/        <- separate git repo: application.yml, <service>.yml, profiles
```

5. Design decisions and patterns (and why)

A **design pattern** is a proven answer to a common problem. Each section below uses the same four questions: the problem, the idea, how we applied it, and the trade-off.

5.1 Centralised configuration (Spring Cloud Config)

Problem. Twelve services each need settings (database URL, ports, feature flags). Copying them into every service is error-prone and changing one value means rebuilding everything.

Idea. Keep settings in one git repository; a config server serves them over HTTP at startup.

How it is applied here. Every service has a tiny application.yml with `spring.config.import: configserver:http://localhost:8888`. Shared values live in application.yml; per-service values in <name>.yml; environment-specific values in profile files (application-docker.yml, application-tls.yml). Because it is git, every change is versioned and reviewable.

Trade-off. The config server must start first. The repo must have at least one commit (otherwise: 'None of labels [] found', see chapter 13).

5.2 Service discovery (Eureka)

Problem. Containers start and stop; their addresses change. Hard-coding http://localhost:8086 breaks immediately.

Idea. Services register with Eureka under a name (ORDER-SERVICE). Callers ask for the name and Eureka gives a live address; the client-side load balancer spreads calls.

How it is applied here. The gateway routes to `lb://order-service` and Feign clients use the service name only.

Trade-off. Eureka is one more moving part, and there is a short delay (about 30 s) before new instances are visible.

5.3 API Gateway

Problem. Clients should not know twelve addresses, and security checks should not be copied twelve times.

Idea. One front door. It terminates TLS, validates the JWT, applies routes and blocks anything not meant to be public.

How it is applied here. Public: login, register, GET products/categories/active campaigns, cart. Everything else under /api/v1/** needs a valid token. The path /internal/** is denied outright, so service-to-service endpoints can never be reached from outside.

Trade-off. A bottleneck if under-provisioned; mitigated by running several gateway copies behind a load balancer.

5.4 Stateless authentication with JWT (RS256)

Problem. With server-side sessions every service would need to share session memory.

Idea. auth-service signs a token with a **private** key (RS256 = RSA + SHA-256). Everyone else verifies it with the **public** key, fetched from auth-service's JWKS URL (/.well-known/jwks.json).

Access tokens last 15 minutes; a longer-lived refresh token gets new ones; logout revokes the refresh token.

How it is applied here. Claims carried: `roles`, `customerId`, `userId`. Passwords are stored as BCrypt hashes. After repeated failed logins the account is temporarily locked.

Trade-off. A stolen access token works until it expires, hence the short life. RS256 (asymmetric) was chosen over HS256 (shared secret) so that other services can verify tokens but can never forge them.

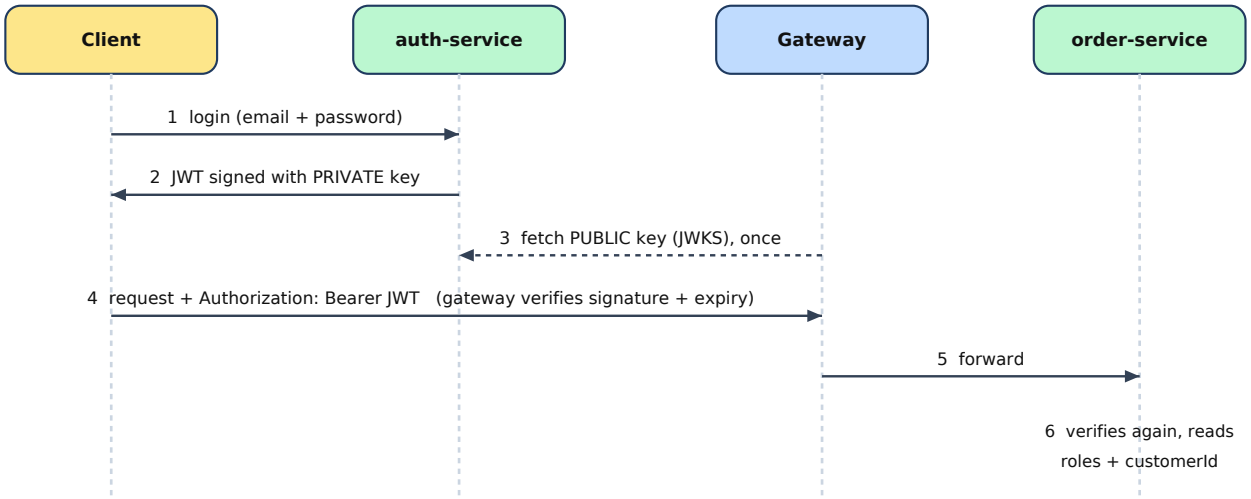


Figure 2 - Token flow: sign once, verify everywhere

5.5 Role-based access control (RBAC)

Problem. A customer must not call admin endpoints or see another customer's order.

Idea. Roles (CUSTOMER, ADMIN) are checked at the endpoint, and ownership is checked in the data query.

How it is applied here. Controllers use method rules like hasRole('ADMIN'). Customer endpoints read customerId from the token (never from the request body) and filter by it, so 'my orders' can only ever return my orders.

Trade-off. Role checks must be kept in sync as endpoints are added; the Postman collection has 'expect 403' tests for this.

5.6 Database per service

Problem. A shared database couples every service: a schema change in one table can break five teams.

Idea. Each service owns its schema and is the only one allowed to touch it. Others use its API or events.

How it is applied here. Twelve MySQL databases (auth_db, order_db...) created by docker/mysql/init-databases.sh. There are no cross-database foreign keys, so services hold plain IDs (productId) instead.

Trade-off. No SQL joins across services and no single transaction across them. This is exactly why the saga (5.9) and outbox (5.10) exist.

5.7 Idempotency

Problem. Mobile networks drop responses. The client retries POST /orders and the customer is charged twice.

Idea. An operation is **idempotent** if doing it twice has the same effect as once. The client sends a unique Idempotency-Key header; the server stores it with the result.

How it is applied here. order-service has a unique constraint on the key. A repeat returns the original order. payment-service is also idempotent per order, so a retried charge request cannot bill twice.

Trade-off. Keys must be stored and eventually cleaned up; the client must reuse the same key when retrying.

5.8 Concurrency control: locks and conditional updates

Problem. Two checkouts can try to take the last unit at the same instant.

Idea. Let the database decide atomically. **Pessimistic lock**: SELECT ... FOR UPDATE makes others wait. **Optimistic lock**: a version column; an update fails if the version changed. **Conditional UPDATE**: `UPDATE stock SET qty = qty - :n WHERE id = :id AND qty >= :n`; if zero rows changed, there was not enough stock.

How it is applied here. Inventory uses row locks and conditional updates; background jobs 'claim' work with a conditional UPDATE so two instances never process the same row; entities carry @Version.

Trade-off. Locks reduce parallelism slightly; keeping them short and inside a small transaction keeps this negligible.

5.9 The Saga pattern (distributed transaction)

Problem. Placing an order touches stock, price, and payment in three different databases. A normal database transaction cannot span them.

Idea. Run it as a sequence of local steps. If a later step fails, run **compensating actions** that undo earlier steps (release the reserved stock).

How it is applied here. order-service orchestrates: price, reserve stock, charge, then commit or release. Reconcilers (scheduled jobs) clean up orders that were interrupted by a crash. Detailed walkthrough in chapter 7.

Trade-off. More code than a single transaction, and the system is briefly in an in-between state (eventual consistency). In return it works across independent services.

5.10 Transactional outbox

Problem. After saving an order you must publish an event to Kafka. If the app crashes between the two, you either lose the event or announce an order that was never saved.

Idea. Write the event into an **outbox table** in the same database transaction as the business change. A separate relay later reads unsent rows and publishes them to Kafka, marking them sent.

How it is applied here. common-lib provides OutboxRecorder (writes the row), OutboxRelay (publishes, retries) and the trace id so tracing continues across Kafka. Consumers handle each message idempotently and failed messages go to a dead-letter topic (DLT).

Trade-off. Delivery is **at-least-once**, so consumers must tolerate duplicates. That is the standard, honest guarantee.

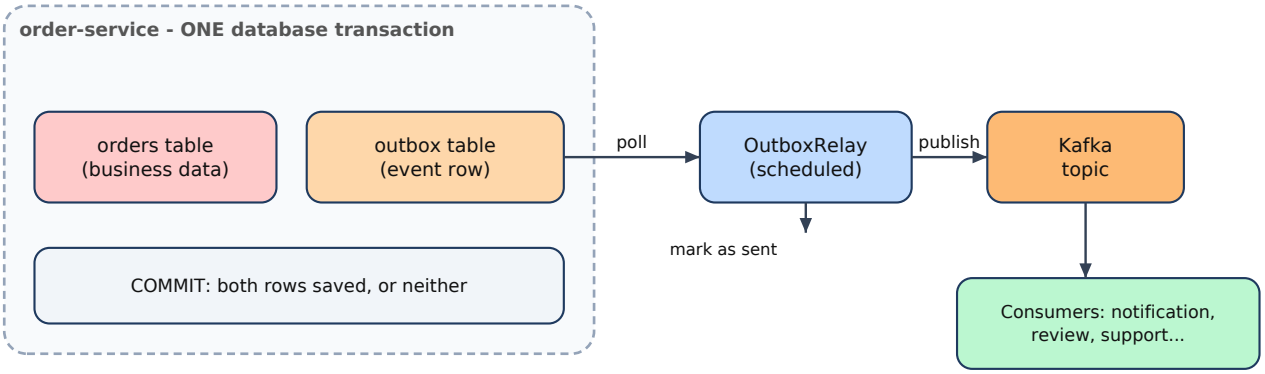


Figure 3 - Transactional outbox

5.11 Resilient synchronous calls (Feign + Resilience4j)

Problem. Order must call inventory, payment, campaign and others. Any of them can be slow or down.

Idea. Feign turns an interface into an HTTP client. Resilience4j adds a 4-second time limit and a circuit breaker per dependency.

How it is applied here. A fallback gives a clear error (HTTP 503 with a message) instead of a hang; the saga then compensates.

Trade-off. Callers must decide what a failure means for the business (for example: a charge timeout is 'unknown', so we reconcile rather than assume failure).

5.12 Soft delete and audit

Problem. Hard-deleting rows destroys history; support and finance need to know what happened.

Idea. Mark rows deleted (deleted_at / status) instead of removing them. Write an audit log of sensitive actions.

How it is applied here. Entities have deleted flags; the Audit API logs auditAction, actor and target on a dedicated AUDIT logger that ELK stores for 200 days.

Trade-off. Queries must always filter out deleted rows; unique indexes need care (the review table uses a generated active_slot column so a deleted review does not block a new one).

5.13 Field-level encryption (AES converter)

Problem. Phone numbers and addresses are personal data. If a database dump leaks, plain text is a breach.

Idea. A JPA AttributeConverter encrypts a field when saving and decrypts when loading, using AES-GCM with a random IV per value.

How it is applied here. The key (AES_KEY) is injected from the environment/Vault, never committed. A unit test proves the same text encrypts to different outputs each time.

Trade-off. Encrypted columns cannot be searched with LIKE; only store encrypted what you do not need to search.

5.14 Shared library (common-lib) and auto-configuration

Problem. Error format, JWT resource-server setup, outbox, audit and the AES converter are needed by many services.

Idea. Put it in one jar. A file named META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports lists the configuration classes; Spring Boot loads them automatically in any service that depends on the jar.

How it is applied here. A service adds the dependency and gets the standard error responses, security and outbox with no extra code.

Trade-off. Changes to common-lib affect everyone, so it is versioned and built first. (A mis-named imports file caused a real startup error: chapter 13.)

6. Implementation: service by service

Each service follows the same recipe: **Controller -> Service -> Repository -> Database**, with Flyway migrations for the schema. For every service below you will see its purpose, its data, its main endpoints, and the interesting logic.

> **Beginner tip**
> All public URLs begin with `/api/v1` and go through the gateway on port 8080. Endpoints that start with `/internal` are for other services only and need the header X-Internal-Key (and a client certificate when TLS is on).

6.1 The anatomy of a typical service

Controller, service and repository (simplified)

```
@RestController
@RequestMapping("/api/v1/customers")
class CustomerController {
    private final CustomerService service;           // injected by Spring

    @GetMapping("/me")
    CustomerResponse me(@AuthenticationPrincipal Jwt jwt) {
        long customerId = CurrentCustomer.id(jwt);   // reads the claim, throws if admin token
        return service.get(customerId);
    }
}

@Service
class CustomerService {
    private final CustomerRepository repo;
    @Transactional(readOnly = true)
    CustomerResponse get(long id) {
        return repo.findByIdAndDeletedFalse(id)
                   .map(CustomerResponse::from)
                   .orElseThrow(() -> new NotFoundException("customer"));
    }
}

interface CustomerRepository extends JpaRepository<Customer, Long> {
    Optional<Customer> findByIdAndDeletedFalse(Long id);   // SQL generated from the name
}
```

Walk-through: the controller does no business thinking; it only extracts the customer id from the verified token. The service wraps the work in a transaction and throws a meaningful exception. The repository method name is itself the query: find by id and deleted = false. A shared exception handler in common-lib converts NotFoundException to a clean HTTP 404 JSON body.

6.2 auth-service (port 8081)

Purpose: own passwords and issue tokens. **Tables:** users_credentials (email unique, password hash, status, failed attempts, locked_until, customer_id), roles, refresh_tokens.

- **POST /auth/login**: finds the user, checks the BCrypt hash, counts failures (lock after the limit), returns access + refresh tokens.
- **POST /auth/refresh**: exchanges a valid refresh token for a new pair (old one is revoked: *rotation*).
- **POST /auth/logout**: revokes the refresh token.
- **GET /auth/.well-known/jwks.json**: publishes the public key for verifiers.

- **AdminBootstrap**: on first start creates admin@retail.local from configuration, so there is someone to manage the system.

Issuing a token (simplified)

```
String issue(User u) {
    Instant now = Instant.now();
    JwtClaimsSet claims = JwtClaimsSet.builder()
        .issuer("retail-auth")
        .subject(String.valueOf(u.getId()))
        .issuedAt(now).expiresAt(now.plus(15, ChronoUnit.MINUTES))
        .claim("roles", u.getRoles())
        .claim("customerId", u.getCustomerId())
        .build();
    return encoder.encode(JwtEncoderParameters.from(claims)).getTokenValue();  // signed with RSA private key
}
```

> **Why we did this**
> BCrypt is deliberately slow and adds a random salt, so stolen hashes are very costly to crack. Rotating refresh tokens means a stolen one stops working the next time the real user refreshes.

6.3 customer-service (8082)

Stores profile and addresses. When registering it creates the customer row and asks auth-service to create credentials; it publishes CUSTOMER_REGISTERED through the outbox so a welcome email is sent. Phone and address lines use the AES converter. Only one address can be default; setting a new default clears the old one inside one transaction.

Field encryption (simplified)

```
@Converter
public class AesAttributeConverter implements AttributeConverter<String, String> {
    public String convertToDatabaseColumn(String plain) {
        byte[] iv = randomBytes(12);                       // new IV every time
        byte[] enc = aesGcm(key, iv).doFinal(plain.getBytes(UTF_8));
        return Base64.encode(iv + enc);                    // store IV with ciphertext
    }
    public String convertToEntityAttribute(String stored) { /* reverse */ }
}
```

6.4 product-service (8083)

Categories (tree) and products. Search combines optional filters (q, categoryId, brand, minPrice, maxPrice, minRating, sort) using a JPA Specification, which builds the WHERE clause dynamically from whichever filters were supplied. GET endpoints are public. It also stores a copy of the average rating (rating_value, rating_count, rating_version) pushed by review-service so product lists can sort by rating without calling reviews each time.

Dynamic search (simplified)

```
Specification<Product> spec = Specification.where(notDeleted());
if (q != null)        spec = spec.and((r, cq, cb) -> cb.like(cb.lower(r.get("name")), "%" + q.toLowerCase() + "%"));
if (brand != null)    spec = spec.and((r, cq, cb) -> cb.equal(r.get("brand"), brand));
if (minPrice != null) spec = spec.and((r, cq, cb) -> cb.ge(r.get("price"), minPrice));
return repo.findAll(spec, pageable);
```

Rating sync and versioning: review-service sends the new average along with a version number. product-service applies it only if the version is newer (a conditional UPDATE). This stops an old, delayed message from overwriting a newer rating.

6.5 inventory-service (8084)

One row per product with quantity and reserved. Operations: set stock (admin), reserve, commit, release, and a history table logging every change with reason and actor. Reservations are keyed by order id so repeating a reserve for the same order does nothing (idempotent).

Atomic reserve (simplified)

```
@Modifying
@Query("update Stock s set s.reserved = s.reserved + :n " +
       "where s.productId = :id and (s.quantity - s.reserved) >= :n")
int reserve(long id, int n);          // returns 0 if not enough stock -> business error 409
```

A reservation also has an expiry. A scheduled job releases reservations whose order never finished (for example the app crashed), so stock is never locked forever. When available stock falls below a threshold a LOW_STOCK event is emitted for the admin email.

6.6 cart-service (8085)

Customers have a cart tied to customerId. Guests get an opaque **X-Cart-Token** returned in the response header on their first add; sending it back identifies their cart. On login, **POST /cart/merge** adds the guest items into the customer cart (summing quantities, capping at stock rules) and removes the guest cart. Prices shown in the cart come from product + campaign-service so discounts appear live.

6.7 campaign-service (8089)

Admins create campaigns (percentage or fixed, start/end, product or category scope). Lifecycle: DRAFT -> SCHEDULED -> ACTIVE -> ENDED/CANCELLED; a scheduled job flips status when time arrives. The **PriceEngine** is a pure function: given a list of lines and the active campaigns it returns the discount per unit; being pure makes it easy to unit test. If several campaigns apply, the best single discount wins (no stacking, to avoid surprising prices).

Price engine core (simplified)

```
BigDecimal discountPerUnit(BigDecimal price, Campaign c) {
    BigDecimal d = (c.type() == PERCENTAGE)
        ? price.multiply(c.value()).divide(BigDecimal.valueOf(100))
        : c.value();
    return d.min(price).setScale(2, RoundingMode.HALF_UP);     // never more than the price
}
```

Note / limitation
Campaign start/end times must be sent in UTC ISO format ending in Z (for example 2026-11-13T09:00:00Z). Mixed time zones were a source of confusion, so the API insists on UTC.

6.8 order-service (8086): see chapter 7

6.9 payment-service (8087)

Simulated gateway. The card **token** decides the result, which makes every failure path testable: tok_ok_4242 succeeds; tok_declined is refused; tok_insufficient means not enough funds; tok_timeout sleeps past the caller's time limit. Each charge is stored once per order id (unique constraint), so a retry returns the earlier result. Supports refund when an order is cancelled after payment.

6.10 review-service (8088)

To create a review the service asks order-service (internal endpoint /internal/orders/purchases) whether this customer has a purchase of the product in a status listed in `retail.review.eligible-statuses` (default DELIVERED). One live review per purchase is enforced by a unique index on a generated column that is NULL for deleted rows. The summary table (count, sum, average) is changed with atomic deltas (count+1, sum+rating) rather than recalculated, so it is fast and safe under concurrency. A scheduled RatingSyncJob retries pushing the rating to product-service every 30 s until acknowledged.

6.11 wishlist-service (8091)

A simple list per customer (name, items with productId and added date). 'Move to cart' calls cart-service and, only on success, removes the wishlist item. Adding the same product twice is ignored, thanks to a unique key on (wishlist, product).

6.12 support-service (8090)

Tickets (subject, category, priority, status OPEN/ASSIGNED/RESOLVED/CLOSED) and messages. Customer endpoints show only their tickets; agent endpoints (/support/agent/**) require ADMIN. Assignment uses a conditional UPDATE (set agent where agent is null) so two agents cannot grab the same ticket. Events SUPPORT_TICKET_CREATED and SUPPORT_TICKET_REPLIED go through the outbox to trigger emails.

6.13 notification-service (8092)

Listens to Kafka and to the internal endpoint /internal/notifications/events. It maps the event type to a **template** (CUSTOMER_REGISTERED, ORDER_CONFIRMED, ORDER_STATUS_CHANGED, SUPPORT_TICKET_CREATED, SUPPORT_TICKET_REPLIED, LOW_STOCK_ALERT), fills placeholders, stores a notification row (PENDING), then a sender job emails it (to MailHog in development). Failures are retried with back-off and visible to admins at /api/v1/notifications, with a manual retry button. Admins can edit template text without redeploying.

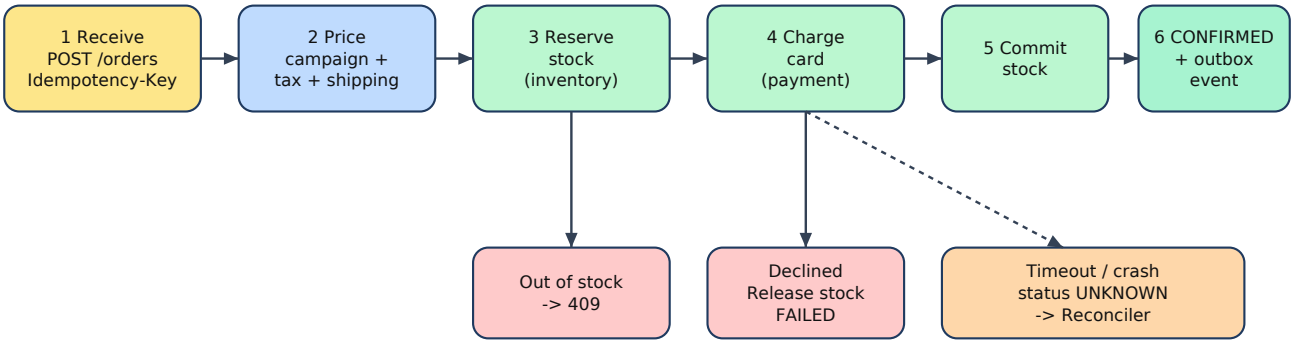
Template rendering (simplified)

```
String render(String template, Map<String,Object> vars) {
    // "Hi {{firstName}}, order {{orderNumber}} is confirmed"
    for (var e : vars.entrySet())
        template = template.replace("{{" + e.getKey() + "}}", String.valueOf(e.getValue()));
    return template;
}
```

7. Deep dive: the order saga

Placing an order is the most delicate flow in the system because it must be correct even when things fail halfway. This chapter follows one order step by step.

7.1 The happy path



Red = failure branches with compensation. Orange = uncertain outcome resolved later by a scheduled reconciler.

Figure 4 - Order saga

1 **Receive.** The controller reads the customer id from the JWT and the Idempotency-Key header. A repeat key returns the existing order immediately.

2 **Price.** Cart lines are priced with campaign discounts. PriceCalculator: subtotal - discount, tax 10%, shipping 5.00 (free when subtotal >= 100). Result stored as an immutable snapshot, so later price changes do not alter this order.

3 **Reserve stock.** inventory-service tries to reserve every line atomically. If any line fails the whole order fails with 409 and nothing is held.

4 **Charge.** payment-service is asked to charge the total with the card token, idempotent per order.

5 **Commit.** Reserved stock becomes sold. The order moves to CONFIRMED and an ORDER_CREATED event is written to the outbox in the same transaction.

6 **Later.** Staff move the order PACKED -> SHIPPED -> DELIVERED; each change is recorded in tracking history and emits ORDER_STATUS_CHANGED.

7.2 Why the database work is split into small transactions

The saga uses Spring's TransactionTemplate to open a short transaction for each local database step, and makes remote calls *outside* any transaction. Holding a database transaction open while waiting for another service would keep locks and a connection busy for seconds, which does not scale and can deadlock. The order row is first saved as PENDING (step 1), then updated after each remote result.

Saga orchestration (simplified)

```
Order o = tx.execute(s -> orders.savePending(cmd));            // short TX #1
try {
    Priced p = pricing.price(o);                               // remote, no TX
    inventory.reserve(o.id(), p.lines());                      // remote, no TX
    PaymentResult r = payment.charge(o.id(), p.total(), token);// remote, no TX
    tx.executeWithoutResult(s -> orders.confirm(o.id(), r));   // short TX #2 (+ outbox row)
} catch (PaymentDeclined e) {
    inventory.release(o.id());                                 // compensation
    tx.executeWithoutResult(s -> orders.fail(o.id(), "DECLINED"));
}
```

7.3 Reconcilers: the safety net

Crashes happen between steps. Scheduled **reconciler** jobs look for orders stuck in an intermediate state for too long, claim them with a conditional UPDATE (so two instances do not double-process), then ask the other service for the truth: 'did you actually charge order 42?'. Based on the answer they either confirm the order or release stock and fail it. Inventory similarly expires old reservations.

7.4 Order status lifecycle

Status	Meaning	Allowed next
PENDING	Saved, saga running	CONFIRMED, FAILED
CONFIRMED	Paid, stock committed	PACKED, CANCELLED
PACKED	Being prepared	SHIPPED
SHIPPED	With courier	DELIVERED
DELIVERED	Received; unlocks reviews	(final)
CANCELLED	Customer/admin cancelled; stock returned, refund issued	(final)
FAILED	Payment/stock failure; nothing charged	(final)

The allowed transitions are enforced in code, so 'DELIVERED -> PACKED' is rejected with HTTP 409/400. The Postman collection tests this by cancelling a delivered order.

7.5 Worked numbers

Line	Calculation	Amount
Item A	19.99 x 2	39.98
Item B	24.50 x 1	24.50
Subtotal		64.48
Tax 10%	64.48 x 0.10 rounded half-up	6.45
Shipping	subtotal below 100	5.00
Total		**75.93**
With 20% campaign on item A	discount 8.00 (20% of 39.98) -> subtotal 56.48, tax 5.65, +5.00	**67.13**

8. Cross-cutting concerns in common-lib

8.1 Uniform error responses

Every service answers errors in the same JSON shape (timestamp, status, error code, message, path, correlation id). A single GlobalExceptionHandler maps exceptions: validation errors to 400, NotFound to 404, conflicts to 409, access denied to 403, unknown to 500. Clients write one error handler for the whole system.

Example error body

```
{ "timestamp":"2026-11-13T09:41:12Z", "status":409, "code":"OUT_OF_STOCK",
  "message":"Only 1 unit of product 7 available", "path":"/api/v1/orders" }
```

Note / limitation
During testing we found that `AccessDeniedException` thrown inside a controller (an admin token calling /customers/me) was caught by the catch-all handler and returned 500 instead of 403. The fix is to add an explicit handler mapping it to 403. The Postman collection accepts either, until that handler is added.

8.2 Input validation

Request DTOs use Jakarta Validation annotations (@NotBlank, @Email, @Size, @Min). Spring rejects bad input with HTTP 400 before any business code runs. This is the first line of defence against bad data.

8.3 Structured logging (ECS JSON) and correlation

Logs are written as JSON in the Elastic Common Schema so Elasticsearch can index fields (service, level, traceId). Each request carries a **trace id**; Micrometer Tracing propagates it through Feign calls and Kafka headers, and Zipkin draws the whole journey of one order across services.

8.4 Audit logging

An AccessAuditFilter and the Audit helper write events like 'ADMIN changed order 42 to SHIPPED' to a special AUDIT logger. Logstash routes them to their own index with a longer retention (200 days) than normal logs (30 days) and traces (7 days).

8.5 Actuator health and metrics

Spring Boot Actuator exposes /actuator/health (used by Docker health checks and CI) and metrics. At the gateway these are admin-only.

9. Production hardening: TLS, secrets, encryption

A system that works on a laptop is not yet safe for production. This chapter covers the four layers added to protect data in transit, at rest and in configuration.

9.1 TLS and mutual TLS between services

TLS encrypts traffic so that nobody on the network can read or alter it (the 's' in https). **Mutual TLS (mTLS)** goes further: both sides present a certificate, so a service can prove *it* is really order-service and the server can refuse any caller without a valid certificate.

Piece	What it is	Where
Private CA	Our own 'certificate authority' that signs all certificates	docker/tls/ca (generated by GenTls.java)
Service certificate	One keystore per service, signed by the CA	mounted in each container at /certs
SSL bundle 'tls'	Spring's named bundle (keystore + truststore)	application-tls.yml in the config repo
JVM argfile	JDK trust/keystore settings so Feign clients also use mTLS	JDK_JAVA_OPTIONS=@/certs/jvm.args
Gateway	HTTPS toward clients and mTLS toward services	GatewayTlsConfig

> **Why we did this**
> The store password is placed in an argfile, not on the command line, so it is not printed in the JVM banner or process list. The Docker overlay file docker-compose.tls.yml switches TLS on without touching the base file.

Test: the smoke test calls a service port directly *without* a certificate and expects the connection to be refused.

9.2 Secrets and encrypted configuration

- **.env file** (never committed): generated by GenEnv.java with random AES_KEY, DB_PASSWORD, CONFIG_PASSWORD, INTERNAL_API_KEY. The generator refuses to overwrite an existing AES_KEY unless --force, because changing it would make already-encrypted data unreadable.
- **{cipher} values**: in the config repository a value can be written as `{cipher}AB12...`. The config server decrypts it with its key before sending to the service, so even the git repo holds no readable secrets.
- **HashiCorp Vault**: a purpose-built secrets store. The config server runs with a *composite* backend: Vault first (order 1), git second (order 2). A secret in Vault overrides the same property in git.

> **Why we did this**
> Secrets in git history are effectively permanent. A secrets manager gives access control, audit and rotation. We chose to read Vault through the config server (instead of every service talking to Vault) so there is one integration point and services remain unchanged.

9.3 Protecting internal endpoints

Endpoints under /internal/** exist only for service-to-service calls. Three independent protections: (1) the gateway denies the path outright; (2) services require the X-Internal-Key header; (3) with TLS on, services require a client certificate. An attacker would need to defeat all three.

9.4 Other security details

- Passwords hashed with BCrypt; tokens short-lived; refresh tokens rotated and revocable.
- CORS and CSRF: pure token-based API with no cookies, so CSRF protection is disabled by design.
- Account lockout after repeated failed logins.
- Principle of least privilege: each service's database user can only reach its own database.
- CI runs a secrets scanner (ci/secrets-scan.sh) and image vulnerability scan (ci/scan-images.sh).

10. Running everything: Docker, Compose and observability

10.1 The Dockerfile

One Dockerfile builds any service using a build argument naming the module. It uses a **multi-stage build**: stage 1 compiles with Maven; stage 2 copies only the jar into a small Java runtime image and runs as a non-root user. Smaller images and a smaller attack surface.

Multi-stage Dockerfile (simplified)

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /src
COPY . .
ARG MODULE
RUN mvn -q -pl ${MODULE} -am -DskipTests package

FROM eclipse-temurin:17-jre
RUN useradd -r app
COPY --from=build /src/${MODULE}/target/*.jar /app.jar
USER app
ENTRYPOINT ["java","-jar","/app.jar"]
```

10.2 docker-compose.yml

Defines the infrastructure (MySQL, Kafka, MailHog, Eureka, config server), the twelve services and the gateway. Key ideas:

- **depends_on + healthcheck**: services wait until MySQL, config server and Eureka are healthy (start order matters: config server first).
- **x-common-env**: a YAML anchor shared by all services so settings are written once.
- **Required variables**: the file refuses to start if AES_KEY, DB_PASSWORD, CONFIG_PASSWORD or INTERNAL_API_KEY are empty (this is the error the PowerShell one-liner produced when $secrets was empty; GenEnv.java fixed it).
- **Profiles**: the heavy ELK/Zipkin tools sit in a profile named observability and start only when asked.
- **Overlays**: docker-compose.tls.yml (mTLS), docker-compose.vault.yml (Vault), docker-compose.deploy.yml (server deployment).

Typical commands

```
# one-time: generate secrets
java GenEnv.java                 # creates .env
java GenTls.java                 # private CA + certificates (optional, for TLS)

docker compose build
docker compose up -d
docker compose -f docker-compose.yml -f docker-compose.tls.yml up -d        # with mTLS
docker compose --profile observability up -d                                 # + ELK + Zipkin
```

10.3 Observability: logs, traces, metrics

Tool	Job	URL (default)
Elasticsearch	Stores and indexes logs	9200
Logstash	Receives JSON logs, routes audit vs normal	5000

Tool	Job	URL (default)
Kibana	Search and dashboards for logs	5601
Zipkin	Distributed trace timelines	9411
MailHog	Catches outgoing emails for inspection	8025 (web), 1025 (SMTP)
Eureka dashboard	Who is registered	8761

Retention policy: normal logs 30 days, audit 200 days, traces 7 days; managed by index lifecycle settings so disks do not fill up.

Beginner tip
Debugging a failed order: copy its trace id from the response or a log line, open Zipkin and paste it. You see every service the request touched and how long each took. In Kibana, search that trace id to read the logs of all those services together.

11. CI/CD: automatic build, test, scan and deploy

Continuous Integration (CI) means every code change is automatically built and tested. **Continuous Delivery/Deployment (CD)** means passing builds are automatically packaged and released. This catches mistakes minutes after they are made instead of weeks later.



Any red step stops the pipeline: broken code never reaches the registry or the server.

Figure 5 - Pipeline stages

Job	What happens	Script
build	Maven compile + unit tests, config repo validation (validate.yml)	ci/build.sh
security	Secret scan; dependency updates via Dependabot; image scan	ci/secrets-scan.sh, ci/scan-images.sh
package	Build the 12 service images + gateway/config/eureka	ci/images.sh
smoke test	Bring the stack up, register a user, log in, check 401s, check the welcome email arrives in MailHog, check mTLS refuses plain callers	ci/smoke-test.sh
push	Publish images to the container registry with a version tag	ci/push.sh
deploy	SSH to the server, pull images, run compose with the deploy overlay, wait for health	ci/deploy.sh, ci/remote-deploy.sh

Why we did this
Logic lives in plain bash scripts under ci/, not inside the workflow YAML, so developers can run exactly the same steps on their own machine to reproduce a CI failure.

12. Testing

12.1 The testing pyramid

Tests are cheaper and faster the lower they are. Have many small tests and a few big ones.

Level	What it checks	Speed	In this project
Unit	One class in isolation (pure logic)	ms	AesAttributeConverterTest, TokenServiceTest, PriceEngine/PriceCalculator tests
Integration	A service with its real database/Spring context	seconds	Spring Boot tests with Flyway migrations (run in CI)
Contract / API	Real HTTP calls through the gateway	seconds	Postman collection (103 requests)
End-to-end smoke	Whole stack in Docker, a few key journeys	minutes	ci/smoke-test.sh

12.2 Unit tests we wrote and what they prove

AesAttributeConverterTest (simplified)

```
@Test
void sameValueEncryptsDifferentlyEachTime() {
    var c = new AesAttributeConverter(KEY);
    assertThat(c.convertToDatabaseColumn("x"))
        .isNotEqualTo(c.convertToDatabaseColumn("x"));   // random IV
}
@Test
void roundTripReturnsOriginal() {
    assertThat(c.convertToEntityAttribute(c.convertToDatabaseColumn("hello"))).isEqualTo("hello");
}
```

TokenServiceTest issues a token and verifies it with the public key, checking claims (roles, customerId, expiry). While running it we met a failure: Spring's getIssuer() tries to parse the issuer as a URL, but our issuer is the plain text 'retail-auth'. The fix is to read it with getClaimAsString("iss") in the test. A test failing for a test-side reason is common and is why we investigate rather than just change the code.

12.3 Testing with Postman (manual and automated)

The delivered collection **retail-microservices.postman_collection.json** and environment **retail-local.postman_environment.json** exercise every feature through the gateway.

1. Import both files; select the 'Retail - local Docker' environment.
2. Start the stack: `docker compose up -d` and wait until services are healthy.
3. Open the collection, click Run, keep the folder order. Folder 00 logs in as admin and a freshly registered customer and stores tokens in variables.
4. Each request has test scripts like `pm.test('status is 200', ...)`. Green ticks mean the behaviour matches expectations.
5. Tokens expire after 15 minutes: re-run folder 00 if you see 401.
6. With TLS on, change baseUrl to https://localhost:8080 and trust docker/tls/ca/ca.crt (or disable SSL verification in Postman settings).

Folder	Key checks
00 Setup & Auth	login, register, refresh, JWKS
01 Gateway security	401 without/with bad token, /internal blocked, 403 for wrong role
02 Customers	profile, address, duplicate email 409, weak password 400
03 Catalog	admin CRUD, public reads, search, customer cannot create (403)
04 Inventory	set stock, history
05 Campaigns	create/publish/active, discounted price, cancel
06 Cart	guest token, merge, update, invalid quantity 400
07 Orders & payments	checkout, idempotent replay, status flow, declined/insufficient/timeout tokens, cancel
08 Reviews	create after delivery, summary, moderation, validation
09 Wishlist	add, move to cart, remove
10 Support	customer ticket flow and agent flow
11 Notifications	admin list/retry/templates; MailHog inbox
12 Internal	direct internal endpoints with X-Internal-Key
13 Cleanup	cancel campaign, logout, refresh after logout fails

12.4 Testing failure paths on purpose

Good systems are judged by how they fail. Use the special payment tokens: **tok_declined** should leave stock untouched and no charge; **tok_timeout** exercises the 4-second limit and the reconciler; stop the payment container while checking out to see the circuit breaker return 503 and the saga release stock. Replay the same Idempotency-Key to prove no duplicate order appears.

12.5 What to check in the data

- inventory history shows reserve then commit (or reserve then release) for each order.
- order tracking lists each status change with a timestamp.
- outbox table rows end up marked sent; none stuck for long.
- MailHog (http://localhost:8025) shows the welcome and order emails.
- Kibana contains an AUDIT entry for every admin action.

13. Real problems we hit (and what they teach)

These are actual errors from this project's development. Reading them is the fastest way to learn how the pieces fit.

Symptom (log message)	Cause	Fix / lesson
ConfigClientFailFastException : None of labels [] found (gateway)	The config repository folder had files but the git repo had no usable commit/branch, so the config server had nothing to serve.	git init, add the yml files, commit. The config server reads committed content only; uncommitted files are invisible.
No spring.config.import property has been defined (auth-service)	Spring Boot 3 no longer uses bootstrap.yml; the client must declare where config comes from.	Add spring.config.import: configserver:http://localhost:8888 (or optional:configserver: to make it non-fatal).
Unable to read meta-data for class CommonLibAutoConfiguration.txt	The auto-configuration list file had the wrong name, so Spring tried to treat 'CommonLibAutoConfiguration.txt' as a class.	Name must be exactly META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports, one class name per line. Rebuild common-lib with mvn install.
Duplicate entry 'admin@retail.local' for key uk_users_email	AdminBootstrap tried to insert the admin even though it existed (a check-then-insert race / second runner).	Make bootstrap idempotent: check existence in the same step, and catch DataIntegrityViolationException. Unique constraints did their job by protecting the data.
TokenServiceTest: Unable to convert claim 'iss' ... to URL	Test called getIssuer() (URL type) while issuer is plain text.	Use getClaimAsString("iss") or make the issuer a URL consistently.
AccessDeniedException: Not a customer account (500)	Admin token has no customerId; the handler turned the exception into 500.	Add a handler mapping AccessDeniedException to 403 (see 8.1).
Flyway: MySQL 26.10 newer than this version	Warning only; the local MySQL version string is newer than Flyway has tested.	Safe to ignore for development; upgrade Flyway when a newer release supports it.
required variable AES_KEY is missing a value	The .env file was written without the generated secrets because the PowerShell variable was empty.	Use GenEnv.java (pure JDK) to create .env reliably.
JwtConfig: No JWT key configured: generating temporary RSA key pair	Dev convenience warning: tokens stop working on restart.	Configure retail.jwt private/public keys (Vault) outside development.

Beginner tip
When something does not start: (1) read the **first** exception and the 'Caused by' at the bottom, not the 100 lines of stack trace between them; (2) ask 'which part of the chain failed: config, database, discovery, security?'; (3) check the config server is up and the repo is committed.

14. Step-by-step: run it yourself

14.1 Prerequisites

- JDK 17 or newer, Maven (or the Maven wrapper), IntelliJ IDEA
- MySQL 8 (for IDE runs) or Docker Desktop (for the full stack)
- Git, Postman

14.2 Option A: run from IntelliJ (development)

1 Create the config repo: `cd D:\retail-config-repo; git init; git add .; git commit -m init`.

2 Build everything: `mvn clean install` from the parent folder (common-lib first).

3 Start in order: config-server, eureka-server, then auth-service and the others, then api-gateway last.

4 Open http://localhost:8761 and wait until each service shows UP.

5 Open Postman and run the collection (baseUrl http://localhost:8080).

14.3 Option B: Docker Compose (whole stack)

1 Generate secrets: `java GenEnv.java`.

2 Build images: `docker compose build`.

3 Start: `docker compose up -d`; watch with `docker compose ps`.

4 Optional: add TLS overlay, Vault overlay, or `--profile observability`.

5 Default admin login: admin@retail.local with the password from .env (development default Admin@12345 if you did not set one).

14.4 Handy URLs

What	URL
API (via gateway)	http://localhost:8080/api/v1/...
Eureka	http://localhost:8761
Config for a service	http://localhost:8888/order-service/default
MailHog inbox	http://localhost:8025
Kibana	http://localhost:5601 (observability profile)
Zipkin	http://localhost:9411 (observability profile)

15. Honest limitations and next steps

> **Note / limitation**
> Be clear about what has and has not been verified. This was built step by step with the developer; individual services were reviewed through logs and screenshots, but a full end-to-end run of every feature on the final Docker stack has not been confirmed in this guide. Treat the items below as a to-do list, not as defects hidden from you.

- **Run the whole Postman collection** and fix any DTO field-name mismatches (expect a few 400s the first time).
- **Product-service category POST** returned HTTP 500 during development and its root cause was not yet diagnosed. Check the service log's stack trace; common culprits are a missing column in the Flyway migration or a date/time format (use UTC ISO with Z).
- **Add 400/404/405 handlers** in common-lib so malformed JSON, wrong HTTP method and unknown paths return clean JSON errors; map AccessDeniedException to 403.
- **Tooling versions to double-check on your machine:** Vault image and CLI output formats, spring-vault-core and feign-micrometer versions in the BOM, Elasticsearch/Zipkin image tags, relaxed-binding environment variable names, and hostname verification in Reactor Netty. These were reasoned from documentation and may need small adjustments.
- **Payment is simulated.** A real provider (for example Stripe) needs webhooks, 3-D Secure and PCI considerations.
- **Not yet included:** load testing, contract tests (Pact), rate limiting at the gateway, a front-end, Kubernetes manifests, backup/restore procedures.
- **Learning path:** read order-service first (it uses most patterns), then common-lib, then try breaking things on purpose (stop a container, send a duplicate key) and watch the logs.

16. Glossary

Term	Plain meaning
API	A set of URLs/functions other programs can call
API Gateway	Single front door that checks identity and routes requests
At-least-once delivery	A message is delivered one or more times, never lost; duplicates possible
Auto-configuration	Spring Boot setting things up automatically when a library is present
BCrypt	Slow, salted password-hashing algorithm
BOM	Bill of materials: a list of compatible library versions
Circuit breaker	Stops calling a failing service for a while
Compensating action	An action that undoes an earlier step (release stock)
Container / image	Running copy / packaged template of a program
DLT	Dead-letter topic: parking place for messages that keep failing
DTO	Object shaped for sending over the network
Eventual consistency	Data becomes consistent shortly after, not instantly
Feign	Library that turns a Java interface into an HTTP client
Flyway	Tool that applies versioned SQL migrations
Idempotent	Safe to repeat: same result as once
JPA / Hibernate	Object-to-table mapping standard / its implementation
JWT / JWKS	Signed login token / published public keys to verify it
Kafka	Durable event log used for messaging
Microservice	Small independently deployable program owning one business area
mTLS	TLS where both sides present certificates
Optimistic / pessimistic lock	Detect conflicts after (version) / prevent them by waiting (row lock)
Outbox	Event row written in the same transaction as data, published later
RBAC	Permissions granted by role
Reconciler	Scheduled job that repairs half-finished work
REST	Style of web API built on URLs and HTTP verbs
RS256	JWT signing with RSA private key, verified with public key
Saga	Chain of local transactions with compensations
Service discovery	Finding live service addresses by name
Soft delete	Mark as deleted instead of removing the row
Stateless	Server keeps no per-user memory between requests
TLS	Encryption of network traffic
Vault	Tool for storing and handing out secrets securely

End of guide.